

INFORMATIQUE 2

Algorithmique 2 / Python

Chapitre 6 : Complexité

Par:

Pr. Mohamed-Amine Chadi

Email: m.chadi@uca.ac.ma

Plan:

1. Motivation
2. Définition de complexité
3. Comment mesure-t-on la complexité, notation grande $O()$
4. Les types de complexités
5. Exemples
6. Règles de simplifications (complexités imbriquées)
7. Exemples

Motivation

Pourquoi comparer plusieurs algorithmes

Nous avons vu jusqu'à maintenant plusieurs algorithmes

- tri
- recherche
- calcul de factoriel
- vérifier si palindrome
- etc.

Aussi, nous avons vu différents choix d'approches d'implémentation

- simple
- itérative
- récursive
- etc. (existent d'autres que nous n'avons pas vu dans ce cours)

Aussi, on peut utiliser différentes structures de données pour la même tâche

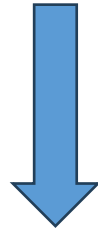
- liste
- set
- tuple
- dictionnaire
- etc.

Pourquoi comparer plusieurs algorithmes

Pourquoi on a étudié tout ça ?

Parce que différents choix (itérative ou récursive, listes ou dictionnaire, etc.) donneront différentes complexités algorithmiques

Qu'est-ce que complexité ??



Définition

Complexité : c'est quoi ?

La complexité est un domaine de l'informatique qui analyse les algorithmes en fonction des ressources nécessaires à leur exécution.

Le nombre de ressources nécessaires varie en fonction de la taille de l'entrée, donc la complexité est généralement exprimée comme une fonction de n , où n est la taille de l'entrée.

Il existe deux types de complexité : la **complexité temporelle** et la **complexité spatiale**.

- La complexité temporelle décrit la quantité de temps nécessaire pour exécuter un algorithme. La complexité temporelle est généralement estimée en comptant le nombre d'opérations élémentaires effectuées par l'algorithme, en supposant que chaque opération élémentaire prend un temps fixe pour s'exécuter.
- La complexité spatiale est la quantité d'espace mémoire nécessaire pour résoudre un problème par rapport à la taille de l'entrée.

Bien que la complexité spatiale soit importante lors de l'analyse d'un algorithme, dans cette histoire, nous nous concentrerons uniquement sur la **complexité temporelle**.

Comment mesure-t-on la complexité temporelle

3 étapes principales:

1. Compter le nombre d'opérations fondamentales
2. Laisser que le terme de croissance la plus rapide
3. Eliminer les coefficients

Complexité temporelle

La complexité temporelle est comment le temps d'exécution varie en fonction de la taille de l'entrée.

Il se peut que certaines personnes aient des ordinateurs plus puissants que d'autres (avec plus de processeurs et de rapidité etc.)

Ce qui va résulter donc en un temps d'exécution plus rapide

Mais ce n'est pas ça ce qui nous intéresse dans ce cours,

La complexité temporelle mesure le temps d'exécution indépendamment de la puissance de la machine

Précisément, **la complexité temporelle est comment le nombre d'instructions fondamentales (addition, soustraction, multiplication, division, affectation, comparaison, etc.) varie en fonction de la taille de l'entrée**

Complexité temporelle

Supposons que nous ayons la liste non triée suivante [1, 5, 3, 9, 2, 4, 6, 7, 8] et que nous devons trouver l'index d'une valeur dans cette liste en utilisant la recherche linéaire.

Meilleur cas : il s'agit de la complexité pour résoudre le problème avec la meilleure entrée possible. Dans notre exemple, le meilleur cas serait de rechercher la valeur 1. Comme il s'agit de la première valeur de la liste, elle serait trouvée dès la première itération.

Cas moyen : il s'agit de la complexité moyenne pour résoudre le problème. Dans notre exemple, nous pourrions dire que le cas moyen serait lorsque nous recherchons une valeur au "milieu" de la liste, par exemple, la valeur 2.

Pire cas : il s'agit de la complexité pour résoudre le problème avec la pire entrée possible de taille n . Dans notre exemple, le pire cas serait de rechercher la valeur 8, qui est le dernier élément de la liste.

Généralement, lorsque nous décrivons la complexité temporelle d'un algorithme, nous parlons du **pire cas**.
Pour décrire la complexité temporelle d'un algorithme, nous utilisons une notation mathématique asymptotique appelée « **Grande-O** » ou « **Big O** ».

Exemple

Quelle est la complexité de cette fonction ?

```
## O(1)

def fct(c):
    return c*2/5 + 4    #combien d'opérations ici ?
                        #==> 3 (multiplication, division, et addition)
                        # mais c'est toujours 3 instructions, quel que soit 'c'

fct(5)

6.0
```

Dans cet exemple, quel que soit la taille de la liste d'entrée, le nombre d'instructions à exécuter sera toujours le même,

C'est ce qu'on appelle une **complexité constante**, et en notation grande O, on l'écrit sous le symbole **O(1)**

Exemple

Quelle est la complexité de cette fonction ?

Dans cet exemple, le nombre d'instructions à exécuter varie de manière proportionnelle avec la taille de l'entrée 'n',

C'est ce qu'on appelle une **complexité linéaire**, et en notation grande O, on l'écrit sous le symbole **$O(n)$**

```
## O(n)

def somme(n):
    total = 0           # 1op
    for i in range(n+1): # 1op
        total += i      # 2op (affectation et addition)
    return total        # 1op

## combien d'opérations ? ==> 1 + (1+2)*n + 1 = 2 + 3n
```

Les règles de calcul (3 étapes principales):

1. Compter le nombre d'opérations fondamentales = $2 + 3n$
2. Laisser que le terme de croissance la plus rapide = $3n$
3. Eliminer les coefficients = n

$O(n)$

Types (ou catégories) de complexité

Les différents types de complexité

Constante	$O(1)$
-----------	--------

Logarithmique	$O(\log n)$
---------------	-------------

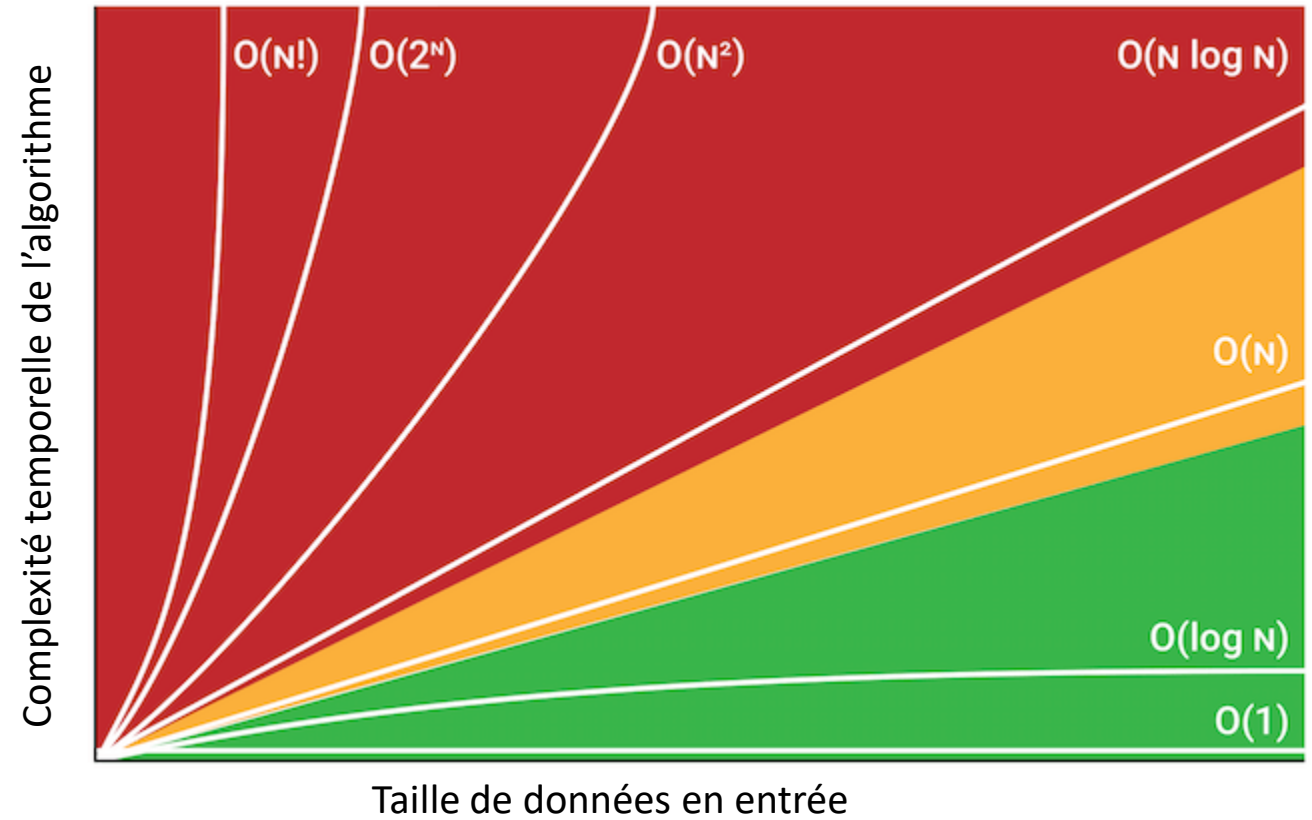
Linéaire	$O(n)$
----------	--------

Quasilinéaire	$O(n \log n)$
---------------	---------------

Quadratique	$O(n^2)$
-------------	----------

Exponentielle	$O(2^n)$
---------------	----------

Factorielle	$O(n!)$
-------------	---------



De plus on se rapproche de $O(1)$, de plus notre algorithme est mieux

**Exemple pour chaque catégorie
de complexité**

Complexité constante, $O(1)$

Un algorithme est dit à temps constant lorsqu'il ne dépend pas de la taille de l'entrée (n). Quelle que soit la taille des données d'entrée, la durée d'exécution sera toujours la même.

Par exemple:

```
def affiche_premier(data):  
    return data[0]  
  
valeurs = [10, 5, 7, 3, 4, 9, 6, 13, 8, 22]  
print(affiche_premier(valeurs))
```

10

Complexité logarithmique, $O(\log(n))$

- On dit qu'un algorithme a une complexité temporelle logarithmique lorsqu'il réduit la taille des données d'entrée à chaque étape (il n'est pas nécessaire d'examiner toutes les valeurs des données d'entrée), par exemple:
 - Taille initiale $n = 32$,
 - à $i=1$, $n = 16$
 - à $i=2$, $n = 8$
 - à $i=3$, $n = 4$, etc.
- Prenons l'exemple de la recherche binaire (ou dichotomique) que nous avons déjà vu dans la séance précédente.

```
def recherche_binaire(arr, cible):  
    bas, haut = 0, len(arr) - 1  
    while bas <= haut:  
        milieu = (bas + haut) // 2  
        if cible == arr[milieu]:  
            return milieu # Indice de l'élément cible  
        elif cible > arr[milieu]:  
            bas = milieu + 1  
        else:  
            haut = milieu - 1 ## cible < arr[milieu]  
    return -1 # Cible non trouvée
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  
res = recherche_binaire(a, 20)  
res
```

6

Complexité logarithmique, $O(\log(n))$

```
def recherche_binaire(arr, cible):
    bas, haut = 0, len(arr) - 1
    while bas <= haut:
        milieu = (bas + haut) // 2
        if cible == arr[milieu]:
            return milieu # Indice de l'élément cible
        elif cible > arr[milieu]:
            bas = milieu + 1
        else:
            haut = milieu - 1
    return -1 # Cible non trouvée
```

```
a = [3, 7, 9, 10, 12, 16, 20, 32]
res = recherche_binaire(a, 20)
res

6
```

Nous avons commencé par une liste de 8 valeurs, après la première itération, la taille de la liste où nous devons chercher est devenue 4, après la taille est devenue 2, et nous avons trouvé la valeur que nous cherchons.

```
a = [3, 7, 9, 10, 12, 16, 20, 32]  ## bas = 0, haut = len(a)-1 = 7
                                   ## comparer 20 avec le nombre en milieu, (bas + haut) // 2 = 3
                                   ## 20 == a[3] ? non
                                   ## 20 > a[4] ? oui --> chercher dans la moitié droite

a = [3, 7, 9, 10, 12, 16, 20, 32]  ## bas = milieu + 1 = 3+1 = 4 (moitié droite = 4 à 7)
#a = [12, 16, 20, 32]             ## milieu = (bas + haut) // 2 = (4+7) // 2 = 11//2 = 5
                                   ## 20 == a[5] ? Non
                                   ## 20 > a[5] ? oui --> chercher dans la moitié droite

a = [3, 7, 9, 10, 12, 16, 20, 32]  ## bas = milieu + 1 = 5+1 = 6 (moitié droite = 6 à 7)
#a = [20, 32]                     ## milieu = (bas + haut) // 2 = (6+7) // 2 = 13//2 = 6
                                   ## 20 == a[6] ? Oui --> cible trouvé

## et donc nous l'avons trouvé dans 3 étapes seulement
## en comparaison avec la recherche linéaire qui compare tous les éléments de la liste
```

Complexité linéaire, $O(n)$

On dit qu'un algorithme a une complexité temporelle linéaire lorsque le temps d'exécution augmente au plus linéairement avec la taille des données d'entrée. C'est la meilleure complexité temporelle possible lorsque l'algorithme doit examiner toutes les valeurs des données d'entrée.

```
valeurs = [1, 2, 3, 4, 5, 6, 7, 8, 9]
for value in data:
    print(value)      # 1op * n
```

1
2
3
4
5
6
7
8
9

```
def recherche_lineaire(arr, cible):
    for i in range(len(arr)):
        if arr[i] == cible:
            return i          # Indice de l'élément cible
    return "cible non trouvée" # Cible non trouvée
```

```
data = [1, 2, 9, 8, 3, 4, 7, 6, 5]
print(recherche_lineaire(data, 9))
```

2

Complexité quasi-linéaire, $O(n \log(n))$

- Un algorithme est dit avoir une complexité temporelle quasi-linéaire lorsque chaque opération dans les données d'entrée a une complexité temporelle linéaire, et ceci se répète $\log(n)$ fois.
- Par exemple, le tri par fusion se compose de deux étapes principales: la division et le tri-fusion.
- **Le tri ici nécessite n itérations, et ceci est pour chaque $\log_2(n)$ division**

```
def tri_fusion(arr):
    if len(arr) > 1:
        milieu = len(arr) // 2
        moitie_gauche = arr[:milieu]
        moitie_droite = arr[milieu:]

        tri_fusion(moitie_gauche)    ## appel récursif
        tri_fusion(moitie_droite)    ## appel récursif

        i = j = k = 0

        while i < len(moitie_gauche) and j < len(moitie_droite):
            if moitie_gauche[i] < moitie_droite[j]:
                arr[k] = moitie_gauche[i]
                i += 1
            else:
                arr[k] = moitie_droite[j]
                j += 1
            k += 1

        while i < len(moitie_gauche):
            arr[k] = moitie_gauche[i]
            i += 1
            k += 1

        while j < len(moitie_droite):
            arr[k] = moitie_droite[j]
            j += 1
            k += 1

    return arr

a = [12, 11, 13, 5, 6, 7]
b = tri_fusion(a)
print("Liste après tri : ", b)
```

Liste après tri : [5, 6, 7, 11, 12, 13]

Complexité quasi-linéaire, $O(n \log(n))$

```
def tri_fusion(arr):
    if len(arr) > 1:
        milieu = len(arr) // 2
        moitie_gauche = arr[:milieu]
        moitie_droite = arr[milieu:]

        tri_fusion(moitie_gauche)    ## appel récursif
        tri_fusion(moitie_droite)    ## appel récursif

    i = j = k = 0

    while i < len(moitie_gauche) and j < len(moitie_droite):
        if moitie_gauche[i] < moitie_droite[j]:
            arr[k] = moitie_gauche[i]
            i += 1
        else:
            arr[k] = moitie_droite[j]
            j += 1
        k += 1

    while i < len(moitie_gauche):
        arr[k] = moitie_gauche[i]
        i += 1
        k += 1

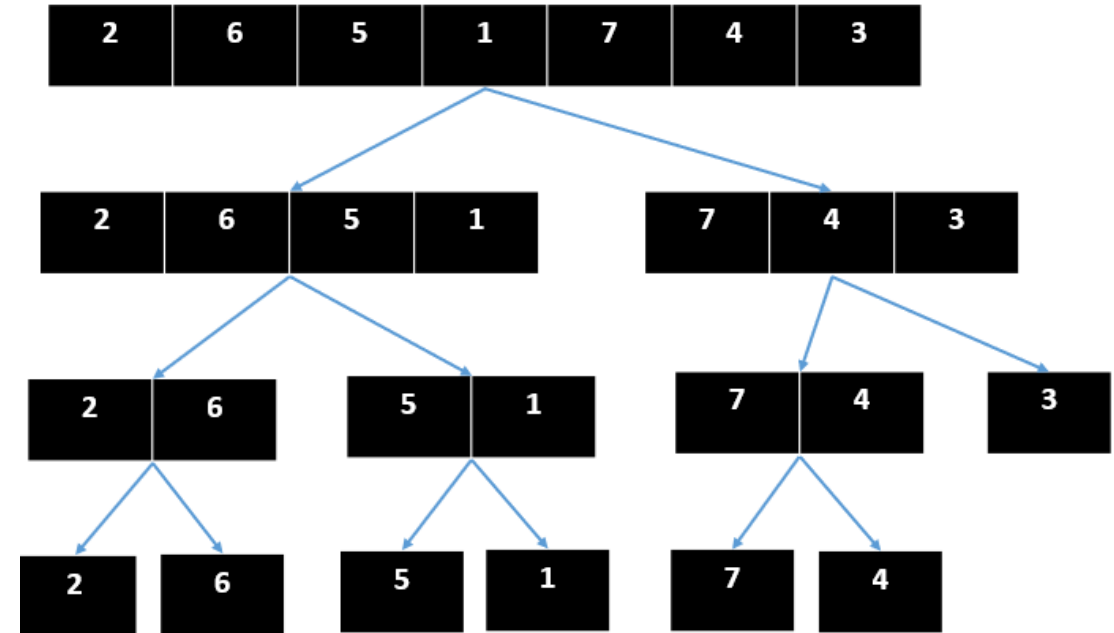
    while j < len(moitie_droite):
        arr[k] = moitie_droite[j]
        j += 1
        k += 1

    return arr

a = [2, 6, 5, 1, 7, 4, 3]
b = tri_fusion(a)
print("Liste après tri : ", b)
```

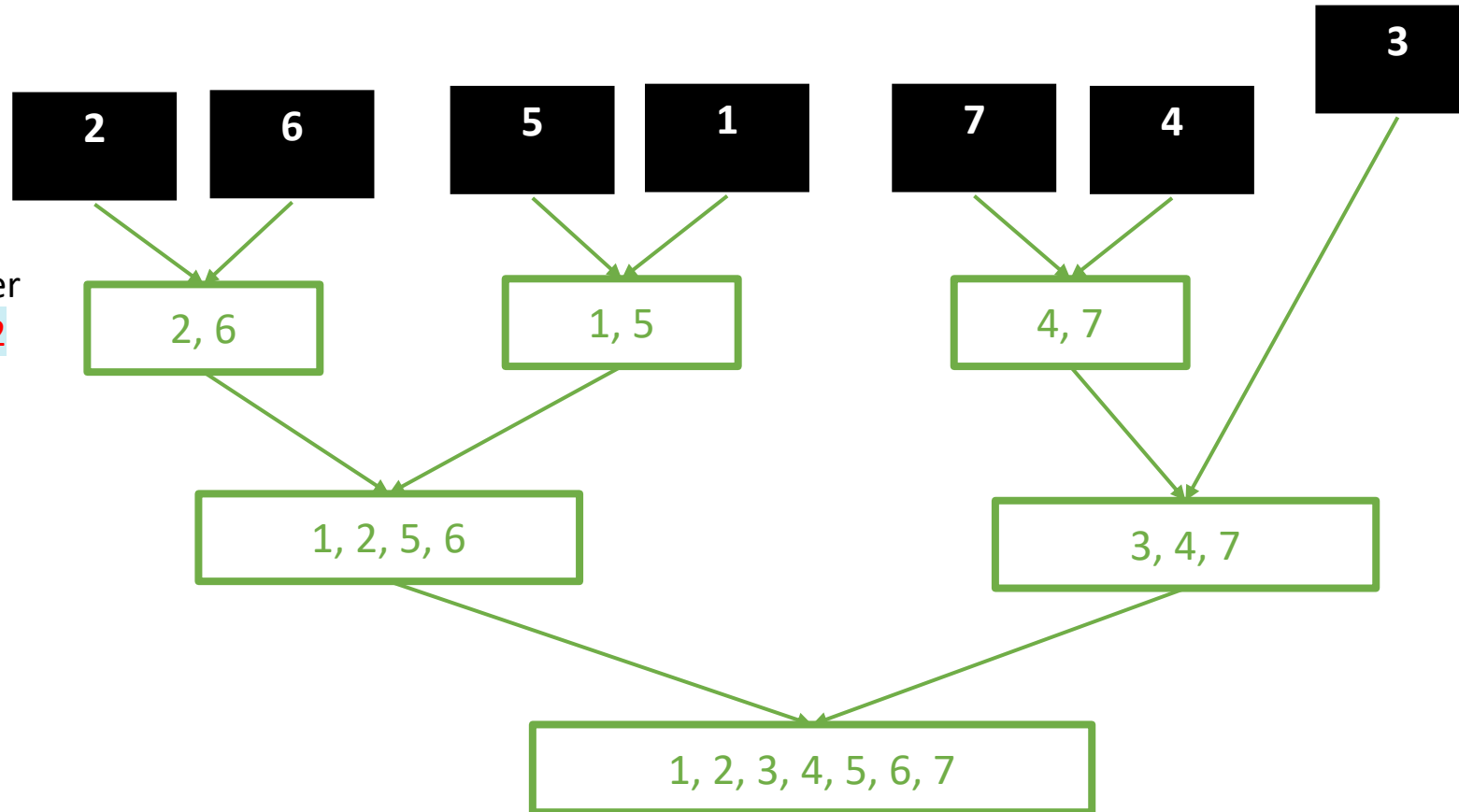
Liste après tri : [1, 2, 3, 4, 5, 6, 7]

On fait le tri-fusion n *fois pour chaque $\log_2(n)$ division



Complexité quasi-linéaire, $O(n \log(n))$

Comparer et affecter
 $2 \text{ op} * (n-1) = 2n - 2$
 $= n$



Pour le tri, on doit itérer sur tous les éléments à comparer (d'où $O(n)$)

Et on fait cette opération de tri $\log(n)$ fois

D'où la complexité $O(n * \log(n))$

```

while i < len(moitie_gauche) and j < len(moitie_droite):
    if moitie_gauche[i] < moitie_droite[j]:
        arr[k] = moitie_gauche[i]
        i += 1
    else:
        arr[k] = moitie_droite[j]
        j += 1
    k += 1

while i < len(moitie_gauche):
    arr[k] = moitie_gauche[i]
    i += 1
    k += 1

while j < len(moitie_droite):
    arr[k] = moitie_droite[j]
    j += 1
    k += 1

```

$$3 * (n - r) + 3 * r$$

r : le reste qui n'a pas été rajouté à arr[k] dans la 1ère while

$3 * n - 3r + 3r$, laissez le terme le plus dominant:
 $3n$, éliminer les coefficients
 , on obtient donc :

$O(n)$

Donc, chaque tri (avec les trois boucles while), nécessite une complexité de $O(n)$

Exemple :

mg : moitié gauche

md : moitié droite

[2] [6] 1ère while, itération 1 : 2<6 ? oui arr[k] = 2 i += 1 k += 1 2ème while, itération 1 : arr[k] = 6 résultat : arr = [1, 6] nbr d'itérations = 2 n = 2	[5] [1] même logique résultat : arr = [1, 6] nbr d'itérations = 2 n = 2	[7] [4] même logique résultat : arr = [4, 7] nbr d'itérations = 2 n = 2	[3] ici, on rentre pas à la 1ère while, car, i < len(mg) est True mais, j n'est pas < len(md) car len(md) = 0 On rentre directement à la 2ème while, 1ère itération : while i < len(mg) : arr[k] = mg[i] i += 1 FIN, car i n'est plus < len(mg) résultat : arr = [3] nbr d'itérations: n = 1
---	---	---	--

Résultat final : nbr d'itérations = 7 = taille totale n ==> donc O(n) pour le tri

Continuité :

[2, 6] [1, 5]

1ère while, _____

itération 1 : *****

2 < 1 ? Non

arr[k] = 1

j += 1

itération 2 : *****

2 < 5 ? oui

arr[k] = 2

i += 1

itération 3 : *****

6 < 5 ? Non

arr[k] = 5

j += 1

FIN 1ère while, car j n'est plus < len(md)

2ème while, _____

itération 1 : *****

arr[k] = 6

résultat :

arr = [1, 2, 5, 6]

nbr d'itérations = 4

[4, 7] [3]

1ère while, _____

itération 1 : *****

4 < 3 ? Non

arr[k] = 3

j += 1

FIN 1ère while, car j n'est plus < len(md)

2ème while, _____

itération 1 : *****

arr[k] = 4

itération 2 : *****

arr[k] = 7

résultat :

arr = [3, 4, 7]

nbr d'itérations = 3

Ok , on vient de comprendre d'où vient le n dans $(n \cdot \log(n))$,

Maintenant, d'où vient $\log(n)$?

Le tri qui prend $O(n)$, se répète $\log(n)$ fois :

ça veut dire, si notre liste comprend 8 éléments, le tri s'effectuera sur 3 sous listes,

exemple :

`a = [2, 6, 5, 1, 7, 4, 3, 8]`

sur :

`[2] [6]    [5] [1]    [7] [4]    [3] [8]`

et puis sur :

`[2, 6] [1, 5]        et    [4, 7] [3, 8]`

et puis sur :

`[1, 2, 5, 6] [3, 4, 7, 8]`

qui donnera la liste triée comme résultat :

`[1, 2, 3, 4, 5, 6, 7, 8]`

Donc, nous avons effectué le tri (qui prend $O(n)$) sur 3 fois qui est $\log_2(8)$ fois

Complexité quadratique, $O(n^2)$

Un algorithme est dit avoir une complexité temporelle quadratique lorsqu'il doit effectuer une opération temporelle linéaire pour chaque valeur des données d'entrée ($n*n$), par exemple :

```
def tri_bulles(lst):  
    n = len(lst)  
    for i in range(n):  
        for j in range(0, n-i-1):  
            if lst[j] > lst[j+1]:  
                # Échanger les éléments si ils sont dans le mauvais ordre  
                lst[j], lst[j+1] = lst[j+1], lst[j]  
  
lst = [64, 34, 25, 12, 22, 11, 90]  
tri_bulles(lst)  
print("Liste triée:", lst)
```

Liste triée: [11, 12, 22, 25, 34, 64, 90]

Complexité exponentielle, $O(2^n)$

Pour l'exemple de la fonction de Fibonacci naïve (que nous avons déjà vu dans le cours de la récursivité, séance 3), où on n'utilise pas la technique de mémorisation.

Pour calculer le terme numéro 3 → on doit faire $2^3 = 8$ instructions

Pour calculer le terme numéro 5 → on doit faire $2^5 = 32$ instructions

Pour calculer le terme numéro n → on doit faire 2^n instructions

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
n_termes = 10    #remplacer ici par 35 pour bien illustrer le temps d'exécution  
  
print("Suite de Fibonacci pour les", n_termes, "premiers termes:")  
  
for i in range(n_termes):  
    print(fibonacci(i), end=" ")
```

```
Suite de Fibonacci pour les 10 premiers termes:  
0 1 1 2 3 5 8 13 21 34
```

Complexité factorielle, $O(n!)$

```
def générer_permutations(arr):
    n = len(arr)
    c = [0] * n
    output = [arr[:]]

    i = 0
    while i < n:
        if c[i] < i:
            if i % 2 == 0:
                arr[0], arr[i] = arr[i], arr[0]
            else:
                arr[c[i]], arr[i] = arr[i], arr[c[i]]
            output.append(arr[:])
            c[i] += 1
            i = 0
        else:
            c[i] = 0
            i += 1

    return output

arr = [1, 2, 3]      #ajoute jusqu'à 9, jupyter notebook stop
permutations = générer_permutations(arr)
print("Permutations possibles de", arr, ":", permutations)
```

$$Pr = \frac{n!}{(n-r)!} = \frac{3!}{(3-3)!} = \frac{6}{1} = 6$$

avec n = taille de la liste
r = taille de sélection

Permutations possibles de [3, 2, 1] : [[1, 2, 3], [2, 1, 3], [3, 1, 2], [1, 3, 2], [2, 3, 1], [3, 2, 1]]

Pour $n = 3$, $n! = 6 < 2^n = 2^3 = 8$, mais des grandes n , exemple :

$$\text{Si } n = 5, \quad Pr = \frac{n!}{(n-r)!} = \frac{5!}{(5-5)!} = \frac{120}{1} = 120 > 2^n = 2^5 = 32$$

Complexités imbriquées: Même règles de calcul

3 étapes principales:

1. Compter le nombre d'opérations fondamentales
2. Laisser que le terme de croissance la plus rapide
3. Eliminer les coefficients

```
## O(1)

def fct(c):
    return c*2/5 + 4    #combien d'opérations ici ?
                        #==> 3 (multiplication, division, et addition)
                        # mais c'est toujours 3 instructions, quel que soit 'c'

fct(5)

6.0
```

```
## O(n^2)

for i in range(n):
    print('a')

for j in range(n*n):
    print('b')

# O(n) + O(n^2) =====> O(n^2)
```

```
n^2 + 2n + 2          =====> O(n^2)
n^2 + 10000n + 3^1000 =====> O(n^2)
log(n) + n + 4        =====> O(n)
0.0001*n*log(n) + 300n =====> O(n*log(n))
2n^20 + 3^n           =====> O(3^n)
```

Exercice

Déterminer la complexité algorithmique de la fonction Fibonacci avec mémorisation


```
memo = {}

def fibonacci_recursive_memo(n):
    ## si le n à calculer est dans notre dictionnaire --> retourne le
    if n in memo:
        return memo[n]

    ## sinon, calcule le, sauvegarde le dans memo, et retourne le
    if n <= 1:
        return n ## résultat pour les indices 0 et 1

    else:
        memo[n] = fibonacci_recursive_memo(n-1) + fibonacci_recursive_memo(n-2)

    return memo[n]
```